

Whole-Contraction JIT Compilation for Numerical Hypervolume in Four Dimensions

Implementation, correctness, and performance report

4 October 2026

Abstract

We apply the compilation strategy of the older **HV4DMagnitude** prefix solver to the newer numerical Chan implementation. Complete term contractions run in Numba, and term states remain native throughout the Python geometric recursion. The geometric cuts and compression schedule are retained. Across five completed matched 4D cases, warm execution is 1.47–8.13 times faster than the earlier hybrid backend, with median ratio 7.76. Both forty-point cases now complete. The older compiled prefix solver remains substantially faster on the larger spherical inputs: compilation removes interpreter and conversion overhead, but not the new algorithm’s intermediate-term growth. We explain the finite-sum implementation, direct magnitude support, correctness checks, timing method, and a separately measured 48.4-second fresh-cache start. No new asymptotic bound is claimed.

1 Purpose and scope

The objective was to keep the new numerical algorithm understandable while moving complete numerical loops into just-in-time (JIT) compiled code. The motivating implementation in **HV4DMagnitude** compiles its complete $2 + 2$ target-plane evaluation [3]. The earlier numerical Chan backend compiled only short array helpers; term construction and merging remained under Python control. Its Numba label therefore did not imply that the expensive algebra was compiled.

The new 4D backend compiles that algebra while preserving the existing recursive partition and scheduled compression [4]. It operates in floating-point arithmetic and supports both ordinary hypervolume and dominated-set ℓ_1 magnitude. All end-to-end measurements in this report are four-dimensional. The generic higher-dimensional backend remains available as the hybrid reference; this report makes no performance claim for it.

Chan’s orthant algorithm provides the theoretical $O(n^{d/3} \text{polylog } n)$ setting, which gives an exponent $4/3$ in four dimensions [1]. The present work changes how numerical operations execute. It neither improves that exponent nor establishes a new global complexity proof for the older prefix implementation.

2 The finite numerical problem

For nonnegative upper corners $A \subseteq \mathbb{R}^4$, write

$$D(A) = \bigcup_{a \in A} \prod_{i=1}^4 [0, a_i].$$

The public interface receives these corners. Minimization points p with reference r are converted to $a = r - p$; both conventions describe the same ordinary hypervolume after reflection.

On each axis, sort the distinct coordinates $0 = c_0 < c_1 < \dots < c_m$. Index zero represents the origin, and index $k > 0$ represents the interval $(c_{k-1}, c_k]$. Box membership is constant on each product cell. Ordinary volume uses the masses

$$w(0) = 0, \quad w(k) = c_k - c_{k-1} \quad (k > 0).$$

For dominated-set magnitude in the anchored ℓ_1 box setting [2], use instead

$$w(0) = 1, \quad w(k) = \frac{1}{2}(c_k - c_{k-1}) \quad (k > 0).$$

This is the product measure with an origin atom and half Lebesgue density on each coordinate axis. The solver stores masses directly. Zero-coordinate generators therefore remain relevant for magnitude even when their ordinary four-dimensional volume is zero.

The state is a finite signed sum of terms. A term has the form

$$T(x) = c \prod_{i \in V} w_i(x_i) \prod_{e \in E} \mathbf{1}_{R_e}(x), \quad (1)$$

where each pairwise predicate is an integer-cutoff condition such as $x_j \leq f(x_i)$ or $x_j \geq f(x_i)$. Cutoff arrays can be increasing or decreasing. Both bound senses and both directions are retained; they cannot in general be collapsed into one staircase per coordinate pair after elimination. Coefficients and intermediate unary arrays may be signed.

In code, a term is simply a named tuple containing a floating coefficient, a native dictionary of unary arrays, and a native dictionary of cutoff arrays. A copied term receives new dictionaries but shares its arrays. Every mask or cutoff update allocates a replacement array. This keeps the representation sparse and prevents one recursive branch from modifying another.

3 One compiled contraction

Here a *contraction* means summing out a variable from a finite product expression. For a fixed assignment z of the remaining variables, collect all bounds on the variable x to be removed. Its feasible interval is

$$L(z) = \max_j \ell_j(z), \quad U(z) = \min_k u_k(z).$$

Constant bounds account for the endpoints of the unary array. Monotonicity allows a cutoff in the opposite orientation to be inverted by a binary search. Each candidate bound depends on at most one remaining variable.

Define the numerical prefix array

$$P(t) = \sum_{0 \leq x < t} w(x), \quad P(0) = 0.$$

If $L \leq U$, the mass to insert into the remaining expression is exactly

$$\sum_{x=L}^U w(x) = P(U+1) - P(L). \quad (2)$$

If $L > U$, the contribution is zero. The code partitions assignments by the first candidate attaining the maximum lower bound and the first candidate attaining the minimum upper bound. Strict comparisons against earlier candidates make these winner regions disjoint, including ties.

On a winner region, each prefix value is a unary function of a remaining variable, or a constant. If the two prefix values depend on different variables, their difference yields two signed terms. If they depend on the same variable, one unary array holds the difference. The required comparisons become masks or monotone pairwise constraints, so expression (1) is retained. Consecutive eliminations implement the numerical pairwise recursion without constructing a dense pair table.

Proposition 1 (Finite-sum preservation). *In exact arithmetic, the contraction described above preserves the sum of the represented expression. Repeated contractions, and block compression formed from them, preserve that sum as well.*

Proof. For each fixed assignment of the remaining variables, the first-winner rule selects exactly one lower winner and one upper winner. If the resulting interval is empty, the original sum and the emitted contribution are zero. Otherwise, equation (2) is the telescoping definition of the prefix array. Multiplying by the unchanged factors and distributing a difference preserves the value. Summing over all assignments proves the first claim. Applying the same equality repeatedly proves the second. The fine-cell blocks form a disjoint partition of the current index interval on each axis. Imposing inclusive block-membership bounds and then eliminating the fine indices therefore preserves total mass when summed over the block indices. $\square$

This proposition explains the algebra, not a rounding-error guarantee for the float64 implementation. For a small magnitude example, let $w = (1, \frac{1}{2}, 1)$, hence $P = (0, 1, \frac{3}{2}, \frac{5}{2})$. The interval $[1, 2]$ has mass $P(3) - P(1) = \frac{3}{2}$, while $[0, 2]$ has mass $P(3) - P(0) = \frac{5}{2}$. The origin atom is included exactly when index zero is included. No integration routine is involved.

Normalization and merging

Each emitted state is normalized by dividing every unary array by its first nonzero entry and moving those factors into the coefficient. Identical states can then be combined. The compiled implementation hashes states in canonical key order and verifies equality by comparing the complete arrays; floating arrays use bitwise comparison, as in the hybrid backend’s byte-string keys. Dictionary insertion order is irrelevant to this equality test, while the candidate-bound order is preserved for tie breaking.

A lookup examines at most sixteen entries in its hash collision chain. If no equal state is found, the new term is retained separately. A missed merge does not discard a contribution, so this limits collision-search work without introducing an approximation. It does not bound total term production or the cost of reading an individual state’s arrays. Cancelled terms are compacted only after emission, keeping chain indices valid throughout merging.

4 The recursion stays clear

The geometric driver maintains a cell, the remaining hard orthants, and the numerical state. First discard boxes disjoint from the current cell. For a remaining box, coordinate i is active when its upper index lies below the cell’s upper index. A box with no active coordinate covers the cell; with one or two active coordinates it is absorbed into a unary mask or pairwise constraint. Boxes with at least three active coordinates remain hard. The following is the essential control flow. At the root, `remaining` is unset. Each recursive step advances the axis and consumes one level of the current block, even when no cut is available.

```
Node(boxes, state, cell, axis, remaining):
    if state is empty: return 0
    hard, state = absorb_easy(boxes, state, cell)
    if state is empty: return 0
    if len(hard) <= 2: return base_sum(...)
    if remaining == 0:
        state, hard, cell = compress_to_blocks(...)
        remaining = unset
    if remaining is unset:
        remaining = restart_block_length(...)
    cut = choose_weighted_median(hard, cell, axis)
    next_axis, next_budget = (axis + 1) % 4, remaining - 1
    if cut is absent:
        return Node(hard, state, cell, next_axis, next_budget)
    left, right = split(cell, axis, cut)
    return Node(hard, state, left, next_axis, next_budget)
        + Node(hard, state, right, next_axis, next_budget)
```

The returned quantity is uncovered mass; the public computation subtracts it from the total mass of the enclosing product grid. At a base cell, at most two hard boxes remain. Their inclusion–exclusion therefore needs at most four rectangle contractions. The complete small-base loop is compiled. The configured base threshold can be changed, but a compiled base with 63 or more hard boxes is rejected to prevent native bitmask overflow.

Compression groups fine cells into consecutive blocks. In 4D it lifts the state to eight variables—four fine indices and four block indices—and imposes the inclusive membership bounds. Eliminating the four fine indices leaves four block indices whose weights represent sums over the old cells. The same mass-summing operation handles signed intermediate states and the magnitude atom. The existing potential-based block schedule, weighted median rule, axis cycling and base threshold are retained.

The subclass introduces only four numerical hooks:

Hook	Responsibility
<code>_initial_terms</code>	Pack the initial state once.
<code>_apply_easy</code>	Apply all classified unary and pairwise updates in Numba.
<code>_base</code>	Evaluate the complete small-base inclusion–exclusion in Numba.
<code>_compress</code>	Sum fine indices into blocks in Numba and retain native output.

Python performs geometric decisions and constructs small mask and block descriptions. It does not unpack the term collection after compression or repack it at every leaf. The geometric recursion is shared with the hybrid backend rather than copied into a second implementation.

5 Why the native state matters

The first compiled version still packed Python term objects into native containers at each base contraction and converted compressed output back into Python objects. Its warm improvement over the hybrid was only 1.12–2.19 times across the five matched cases, with median 1.59. A separate profile of the seed-42 twenty-point sphere exposed the remaining cost:

Profile statistic	Repeated conversion	Persistent native state
Total profiled seconds	2.819	0.265
Python-visible function calls	4,129,144	28,952
<code>pack()</code> calls	41	1
<code>pack()</code> cumulative seconds	2.306	0.001027

Table 1: Independent warm profiles. Native work is not counted as individual Python calls; these counts do not measure arithmetic complexity.

Packing accounted for approximately 81.8% of the first profile. Keeping native term states through the recursion removed the repeated conversion. The final profile spends about 0.175 seconds in base contractions, 0.045 seconds in compression, and 0.030 seconds applying easy constraints. These profiled durations include instrumentation overhead and are not used as benchmark speedups. The change is supported by both the profile and the subsequent independent timing experiment.

6 The 4D timing experiment

Table 2 reports all seven input sets. Each algorithm receives the same saved points. New and hybrid backends receive anchored extents $r - p$; the older prefix and Python Chan implementations receive positive minimization coordinates and reference r . The conversion is included in the new engines’ timed calls. No dominance prefilter is applied, and the base threshold is two where configurable.

Workers run sequentially, with one fresh process per algorithm/input pair. After an un-timed first call, a fresh solver is created for each of three timed calls; their median is

reported. Timed work includes conversion, preprocessing, packing and evaluation. Compilation caches were populated by correctness checks beforehand. Each worker has a 45-second total budget, including imports, first use and all repetitions.

Family seed / n	New JIT	Hybrid	Old prefix	Chan $d/3$	Chan $d/2$
Sphere 42 / 10	4.924	12.331	6.478	9.437	1.408
Sphere 42 / 20	252.106	2050.424	19.284	34.166	5.145
Sphere 42 / 40	7540.933	budget	55.884	134.478	18.260
Sphere 99 / 20	274.465	2172.687	21.673	32.210	5.612
Sphere 99 / 40	6929.245	budget	51.891	122.594	17.727
Grid 7 / 20	3.414	5.013	0.766	2.263	0.362
Uniform 17 / 20	115.610	897.588	14.112	18.851	2.529

Table 2: Warm end-to-end medians in milliseconds. “budget” is a whole-worker timeout; it is not a per-call lower bound. Both Chan reference codes use Python without Numba.

The five completed matched comparisons yield speedups of 1.47–8.13, with median ratio 7.76. For the seed-42 twenty-point sphere, 2.050 seconds becomes 0.252 seconds, an 8.13-fold improvement. Both forty-point spheres complete in the new backend, at 7.541 and 6.929 seconds, whereas the hybrid workers exhaust the budget. No ratio is inferred from those timeouts.

There are 33 completed algorithm/input combinations out of 35, with no numerical mismatches against the independent Python Chan $d/2$ evaluator. Maximum scaled disagreement is 7.89×10^{-16} , using $|x - y| / \max(1, |y|)$. All seven new-backend cases exercised nopython base evaluation and easy updates; the five cases requiring compression also exercised nopython block compression. Fastmath and parallel execution are disabled.

The older compiled prefix solver remains substantially faster on the larger spherical inputs. It takes about 19 milliseconds for seed-42 $n = 20$ and 56 milliseconds for $n = 40$. Python Chan $d/2$ is fastest on all seven inputs. Three warm repetitions on small inputs neither determine asymptotic growth nor establish an eventual crossover; no exponent is fitted here.

The machine is an AMD Ryzen 7 5800H with eight physical cores, sixteen logical processors and approximately 16 GiB RAM, running Windows 11. Software versions are CPython 3.12.14, NumPy 2.5.3 and Numba 0.68.0. Sphere points are absolute Gaussian vectors normalized to unit Euclidean length, with $r = (1.2, 1.2, 1.2, 1.2)$. Seed 42 consumes sizes 5, 10, 20, 40 successively, discarding the five-point case; seed 99 consumes 20, 40. The grid case uses independent integer coordinates in $\{0, 1, 2, 3\}$ divided by four, and the uniform case uses independent coordinates in $[0, 1)$; both have $r = (1, 1, 1, 1)$. All actual inputs are archived. No confidence intervals or processor pinning were used.

First use and compilation

A separate process with an empty dedicated Numba cache measured startup on the seed-42 twenty-point sphere. Imports took 0.803 seconds. The first computation took 48.408 seconds, including compilation and evaluation; the next call took 0.211 seconds. Values and counters agreed, and all three compiled entry points had nopython signatures. These are single observations, not medians, and the first duration is not compiler time alone.

The warm speedup therefore has a substantial first-use cost. Persistent caching is enabled. The cold-start script requires a new or empty cache directory and does not delete existing caches. Its input, source hashes and full measurements are archived separately from the warm benchmark.

7 Correctness and direct magnitude support

The dedicated verification suite checks the numerical kernels against their defining finite sums and compares small complete problems with exact rational inclusion–exclusion. Its main coverage is:

Check	Count
Exact small HV/magnitude instances	50
Signed rectangle contractions	48
Signed easy-constraint cells	16
First-compression cells	144
Masked repeated-compression checks	16
Default twenty-point hybrid comparisons	2
Exact twelve-point cases reaching generation two	2
Affine magnitude/HV comparisons	9

These counts describe different assertions and are not a count of independent large end-to-end instances. Further checks cover empty sets, duplicates, zero coordinates, tied inputs, mixed scales, input immutability, term cancellation, hash collisions, the collision-search cap, native bitmask overflow, public API compatibility, and a fresh-process package import using cached types. All checks pass. Maximum recorded scaled error in the full suite is 2.68×10^{-14} ; this is empirical evidence rather than a universal floating-point error bound.

The native state is packed exactly once in each of the 52 nonempty complete computations instrumented for this invariant, and zero times for the two empty ones. The twenty-point checks perform ten compressions each; the forced twelve-point checks reach a second compression generation. Thus successful tests are not restricted to cases in which compression is bypassed.

Magnitude is also checked through the independent affine identity

$$\text{Mag}(D(A)) = \text{HV}_4(\{(1 + a_1/2, \dots, 1 + a_4/2) : a \in A\}). \quad (3)$$

Every nonempty collection $S \subseteq A$ of anchored boxes has intersection with upper corner $b_i = \min_{a \in S} a_i$. Its product-measure value is $\prod_i (1 + b_i/2)$, which is the ordinary volume of the transformed intersection. Inclusion–exclusion proves (3). This identity concerns anchored-set values; it does not assert translation invariance of the atomic measure.

The reverse direction is equally explicit. If every coordinate of every corner in C is at least one, set $a_i = 2(c_i - 1)$. Then

$$\text{HV}_4(C) = \text{Mag}(D(2(C - \mathbf{1}))).$$

For general nonnegative corners, first discard boxes with a zero extent: they contribute no four-dimensional volume. If no boxes remain, the answer is zero. Otherwise, choose a

positive scale s so that every coordinate of every corner in sC is at least one. Homogeneity gives

$$\text{HV}_4(C) = s^{-4} \text{Mag}\left(D(\{2(sc - \mathbf{1}) : c \in C\})\right).$$

The preprocessing takes linear work. These identities preserve the underlying exact-value computation up to transformations; they do not remove the signed-term expansion or guarantee equal floating-point conditioning.

For the four generators

$$\{(1, 4, 3, 2), (2, 3, 4, 1), (3, 2, 1, 4), (4, 1, 2, 3)\},$$

the public interface returns ordinary HV 69 and magnitude 765/16. The timing table concerns ordinary HV; it does not supply magnitude timing ratios merely because both modes pass correctness checks.

8 What still limits performance

Compilation leaves the number of emitted terms unchanged on the measured matched cases. On seed-42 $n = 20$, both hybrid and new versions use 61 nodes, ten compressions, 10,809 raw terms, 7,441 term eliminations and a peak of 625 terms. The new $n = 40$ run uses 157 nodes, thirteen compressions, 251,360 raw terms, 146,333 eliminations and a peak of 7,501 terms.

The older prefix solver uses a different representation: six staircases and a compiled target-plane scan. It does not perform this scheduled signed-term compression. Its smaller measured cost is consequently not a contradiction of the new backend’s successful compilation. Nor does its speed establish the complete $\tilde{O}(n^{4/3})$ bound for that implementation; a global bound on its target-grid work remains separate from benchmarking.

The useful implementation result is precise: complete contractions can run in native code while the recursion stays short and readable, and retaining native state is essential to realizing the benefit. The remaining target is to reduce or better control the states produced by compression. Changing the schedule, reusing equivalent summaries, or introducing a specialized planar contraction would require its own correctness and complexity analysis. None of those additional improvements is claimed in this report.

9 Use and reproduction

The established 4D Numba entry point now selects the new backend:

```
from numerical_chan4_numba import hypervolume4
value = hypervolume4(corners)
mag = hypervolume4(corners, magnitude=True)
```

Keep the NumericalChanHV4D and NumericalChanHVND directories together. The adapter is `numerical_chan4_compiled.py`; the compiled algebra is `compiled_terms.py`. Both script and package imports use one canonical kernel module name so cached

native tuple types remain importable. Exact arithmetic remains available through `numerical_chan4.py` with suitable integer or rational inputs.

From the repository root, install the requirements in the 4D directory and run `python NumericalChanHV4D/verify_whole_loop.py`. This also populates the cache before warm benchmarking. The benchmark directory `NumericalChanHV4D/benchmarks/whole_loop` contains:

- `benchmark_whole_loop.py`, accepting `-old` with the path to the pinned older HV4DMagnitude checkout;
- `profile_whole_loop.py`, for a separate warm profile;
- `cold_start.py`, accepting `-cache-dir` with a new or empty directory;
- the saved inputs, repetitions, counters, source hashes, profiles and startup record in `results`, and the first compiled version in `results_v1`.

The measured native-state sources are pinned at `1c2b575`; the first compiled version is `7ea05d2` [5]. Release `0c4375e` also selects the new public wrapper and fixes the canonical import path. That import-only change leaves the measured numerical kernel unchanged. The cold-start record uses the release import convention. The independent Chan reference files and older prefix sources are identified by their archived hashes and commits. Full links appear below; this report itself does not require an external BibTeX database or additional figures to compile.

References

- [1] T. M. Chan. *Klee’s Measure Problem Made Easy*. FOCS 2013, pp. 410–419. DOI: 10.1109/FOCS.2013.51. [Author’s August 2013 version](#), especially Sections 4.1–4.2.
- [2] M. T. M. Emmerich. *The Magnitude of Dominated Sets: A Pareto Compliant Indicator Grounded in Metric Geometry*. arXiv:2604.18147, 2026. <https://arxiv.org/abs/2604.18147>.
- [3] M. Emmerich and collaborators. *HV4DMagnitude*, source snapshot `0c668e765202`. [Pinned repository](#). The measured prefix path uses `solver.py`, `prefix4.py` and `fastkernel.py`.
- [4] *FastHVChan / NumericalChanHV4D*, release `0c4375e963c8`. [Code, tests, implementation guide and benchmark data](#). The repository’s Python reference codes are implementations of Chan’s approaches; they are not identified here as code written by Chan.
- [5] *FastHVChan*, numerical source snapshots: [native-state version 1c2b575](#); [first compiled version 7ea05d2](#). Raw data and SHA256 hashes are archived under `benchmarks/whole_loop` in release [4].